

Análisis técnico del proyecto INSPIRA PWA

1) Extracción y contexto

- **ZIP analizado:** `/home/ubuntu/Uploads/inspira_app_pwa.zip`
 - **Directorio de trabajo:** `/home/ubuntu/inspira_app_pwa/inspira_app_pwa`
 - **Tipo de proyecto:** SPA con frontend React y backend serverless en Firebase (Auth + Firestore + Storage).
-

2) Estructura general del proyecto

Raíz

- `package.json` → scripts/build/dependencias
- `vite.config.ts` → configuración Vite + PWA
- `firebase.json` , `.firebaserc` → hosting/proyecto Firebase para CLI
- `firestore.rules` / `firestore-debug.rules` → reglas de seguridad
- `public/manifest.json` , `public/sw.js` → PWA
- `src/` → aplicación principal

src/

- `App.tsx` → orquestador principal (auth, suscripciones, navegación, plan de usuario)
 - `types.ts` → contratos de datos (User, Audio, Book, Speaker, Event, etc.)
 - `constants.ts` → datos mock/fallback (audios, speakers, libros, eventos)
 - `services/`
 - `firebase.ts` → inicialización Firebase
 - `dbService.ts` → servicios Firestore por colección
 - `userService.ts` → lógica específica de usuarios
 - `storageService.ts` → subida de archivos a Storage
 - `components/`
 - `AdminPanel.tsx` → altas/bajas/edición de contenido y usuarios
 - `Home.tsx` , `Books.tsx` , `Calendar.tsx` , `HallOfFame.tsx` , `InstallPWA.tsx` , etc.
-

3) Framework y stack detectado

Framework principal

- **React 19 + TypeScript + Vite 6**
- Confirmado en:
- `package.json`
- `src/main.tsx`

- `@vitejs/plugin-react` en dependencias

UI / utilidades

- Tailwind CSS
 - Motion (`motion/react`)
 - Lucide React
 - Recharts
-

4) Configuración Firebase (Firestore + Storage)

Inicialización runtime (la que usa la app)

Archivo: `src/services/firebase.ts`

- `projectId: 'inspira-bbele'`
- `authDomain: 'inspira-bbele.firebaseio.com'`
- `storageBucket: 'inspira-bbele.firebaseio.com'`
- Inicializa:
- `auth = getAuth(app)`
- `db = getFirestore(app)`
- `storage = getStorage(app)`
- Activa persistencia local de Auth y Firestore (IndexedDB en no-localhost).

Configuración CLI/deploy

- `.firebaserc` apunta a `inspira-app-oficial`
- `firebase.json` hostea carpeta `dist/`

Observación importante: hay **desalineación** entre el runtime (`inspira-bbele`) y el proyecto por defecto de CLI (`inspira-app-oficial`). No rompe lectura/escritura en app web en runtime, pero sí puede causar despliegues/reglas deploy al proyecto equivocado.

Reglas Firestore

Archivo: `firestore.rules`

- Modelo principal:
- `users` : dueño o admin
- `audiobooks/speakers/events/books/editorial_calendar/config/success_paths` : lectura autenticada, escritura admin
- `userPlaylists` : autenticado, update/delete dueño
- Existe archivo permisivo de diagnóstico: `firestore-debug.rules` .

Reglas Storage

- **No se encontró** `storage.rules` en el repositorio.

- Si hay problemas de carga de archivos (fotos/audio), revisar reglas reales de Storage en consola Firebase.

5) Ubicación de archivos clave por dominio funcional

A) Carga de libros / audiolibros

- Firestore CRUD:
- `src/services/dbService.ts`
 - `bookService` (colección `books`)
 - `audioService` (colección `audiobooks`)
- Subida de archivos:
- `src/services/storageService.ts`
 - `uploadAudio()` → `audiobooks/audio/...`
 - `uploadCover()` → `audiobooks/covers/...`
- UI administración:
- `src/components/AdminPanel.tsx`
 - `handleSaveBook`
 - `handleSaveMentoring`
- UI consumo:
- `src/components/Books.tsx`
- `src/components/BookDetail.tsx`
- `src/components/Home.tsx`

B) Mentorías

- Modelo de dato `Audio.contentType = 'mentoring' | 'audiobook' : src/types.ts`
- CRUD/consulta: `audioService` en `src/services/dbService.ts`
- Alta de mentorías (upload + createAudiobook con `contentType: 'mentoring'`): `src/components/AdminPanel.tsx`
- Render en home y biblioteca:
- `src/components/Home.tsx` (filtra por `contentType === 'mentoring'`)
- `src/components/Library.tsx`

C) Startalent (carga de fotos)

- CRUD talentos (`speakers`): `speakerService` en `src/services/dbService.ts`
- Carga de foto talento en admin:
- `src/components/AdminPanel.tsx` (`handleSaveTalent`)
- usa `storageService.uploadCover(...)`
- Render en frontend:
- `src/components/HallOfFame.tsx` (usa `speakers` dinámicos de App)
- `src/components/StarTalent.tsx` y `src/components/StarTalentWall.tsx` (usan `SPEAKERS / MOCK_AUDIOS` de constantes)

D) Eventos y calendario

- CRUD eventos (`events`): `eventService` en `src/services/dbService.ts`
- Alta/edición en admin: `src/components/AdminPanel.tsx`
- Vista calendario usuario:
- `src/components/Calendar.tsx`
- actualmente usa `MOCK_EVENTS` de `src/constants.ts` (no usa props/eventos Firestore)

E) Usuarios gratis vs premium

- Tipos de plan y usuario: `src/types.ts`
- Creación/lectura/actualización:
- `src/services/userService.ts`
- `src/components/Login.tsx` (`ensureUserInFirestore` : crea nuevo usuario con plan `Gratis`)
- `src/App.tsx` (`onAuthStateChanged` y seteo de `userPlan`)
- Gestión manual de planes:
- `src/components/AdminPanel.tsx` (`handleTogglePlan`)
- `src/components/Sidebar.tsx` (simulación visual super admin)

F) PWA y botón de instalación

- Config PWA Vite: `vite.config.ts` (`vite-plugin-pwa` , `injectManifest`)
- Registro SW: `src/registerSW.ts` y `src/main.tsx`
- Manifest: `public/manifest.json`
- Service worker: `public/sw.js`
- Banner + lógica `beforeinstallprompt` : `src/components/InstallPWA.tsx`
- Botón instalar en sidebar: `src/components/Sidebar.tsx` (hook `useInstallPWA`)
- Integración en app: `src/App.tsx` (render `<InstallPWA />` post-login)

G) Firestore: estructura de datos

Fuentes: `src/types.ts` , `src/services/dbService.ts` , `src/services/userService.ts` , `firestore.rules`

Colecciones detectadas:

1. `users`
2. `audiobooks`
3. `books`
4. `speakers`
5. `events`
6. `editorial_calendar`
7. `config` (doc `global`)
8. `payments`
9. `usage_events`
10. `naya_memory`
11. `userPlaylists`

12. `success_paths`
 13. `talent_notifications`
-

6) Mapeo directo a los 7 problemas reportados

Problema 1: “El contenido de los libros no se carga”

Archivos clave

- `src/components/Books.tsx`
- `src/components/BookDetail.tsx`
- `src/services/dbService.ts` (`bookService`)
- `src/components/AdminPanel.tsx` (`handleSaveBook`)

Hallazgos técnicos relevantes

- El módulo de libros depende de colección `books` y de `etapas[].url` para audio por capítulo.
 - Si `etapas[].url` queda null/ausente, aparece como “Próximamente”.
 - En App, el tab Books recibe `books={dynamicBooks.length > 0 ? dynamicBooks : undefined}`, por lo que podría caer en fallback si Firestore no devuelve datos.
-

Problema 2: “No se cargan las mentorías”

Archivos clave

- `src/services/dbService.ts` (`audioService`)
- `src/components/AdminPanel.tsx` (`handleSaveMentoring`)
- `src/components/Home.tsx` (filtro `contentType === 'mentoring'`)
- `src/App.tsx` (suscripción `audioService.subscribeToAudiobooks`)

Hallazgos técnicos relevantes

- Mentoría se identifica exclusivamente por `contentType: 'mentoring'` .
 - Si el contenido se crea sin `contentType` o con valor inconsistente, no aparece en carruseles de mentoría.
 - URLs de audio/preview dependen de Firebase Storage y reglas de acceso.
-

Problema 3: “En Startalent no se cargan las fotos”

Archivos clave

- `src/components/AdminPanel.tsx` (`handleSaveTalent`)
- `src/services/storageService.ts`
- `src/services/dbService.ts` (`speakerService`)
- `src/components/HallOfFame.tsx`
- `src/components/StarTalent.tsx`

- `src/components/StarTalentWall.tsx`

Hallazgos técnicos relevantes

- Se detectan **dos fuentes de datos** para Startalent:
 - 1) Dinámica (`speakers` Firestore) en `HallOfFame.tsx` .
 - 2) Mock (`SPEAKERS`) en `StarTalent.tsx` y `StarTalentWall.tsx` .
- Este split puede causar que fotos nuevas subidas en admin se vean en una vista y en otra no.

Problema 4: “En programa de eventos no se cargan los eventos en el calendario”

Archivos clave

- `src/components/Calendar.tsx`
- `src/services/dbService.ts` (`eventService`)
- `src/App.tsx` (suscribe `events`)

Hallazgos técnicos relevantes

- `Calendar.tsx` consume `MOCK_EVENTS` desde constantes, no consume `events` de Firestore.
- Aunque App suscribe `eventService.subscribeToEvents` , esos datos no se usan en el componente calendario actual.
- Este punto explica de forma directa por qué en producción pueden no verse eventos reales.

Problema 5: “Nuevos usuarios no diferencian gratis/premium”

Archivos clave

- `src/components/Login.tsx`
- `src/App.tsx`
- `src/services/userService.ts`
- `src/components/AdminPanel.tsx` (`handleTogglePlan`)

Hallazgos técnicos relevantes

- Alta normal crea `plan: 'Gratis'` .
- Existen múltiples rutas que fuerzan/alteran plan:
- Super admin/domain admin en `App.tsx` .
- Talento asociado (`isTalent`) puede promover a Premium en ciertos flujos.
- Toggle manual en AdminPanel.
- Simulación visual en Sidebar para superadmin.
- Se recomienda revisar consistencia entre `user.plan` (fuente de verdad) y estados derivados (`userPlan`) y condiciones automáticas.

Problema 6: “En nuevos usuarios no aparece botón descargar app”

Archivos clave

- `src/components/InstallPWA.tsx`
- `src/components/Sidebar.tsx`
- `src/App.tsx`
- `vite.config.ts`, `public/manifest.json`, `public/sw.js`, `src/registerSW.ts`

Hallazgos técnicos relevantes

- El botón solo aparece si `beforeinstallprompt` está disponible (`canInstall === true`).
- Se oculta si:
- app ya instalada,
- usuario pospuso (cooldown 7 días guardado en `localStorage`),
- navegador no soporta prompt (iOS Safari),
- condiciones PWA no satisfechas por contexto.
- Hay doble punto de entrada: banner inferior y botón en sidebar, ambos comparten el mismo prompt cacheado.

Problema 7: “Borrar contenido de prueba (TEST*, libros/audios/eventos de prueba)”

Archivos clave

- `src/components/AdminPanel.tsx` (borrado unitario manual)
- `audioService.deleteAudio`
- `bookService.deleteBook`
- `eventService.deleteEvent`
- `speakerService.deleteSpeaker`
- `src/services/dbService.ts` (métodos de delete por colección)

Hallazgos técnicos relevantes

- No existe script dedicado de limpieza masiva por patrón (`TEST*`) en el repo.
- La limpieza actual está orientada a borrado item por item desde UI admin.
- Se requeriría script operativo (Node/Firebase Admin SDK) o módulo admin para borrado por regex/ prefijo para ejecutar limpieza completa de datos de prueba.

7) Estructura de datos Firestore (resumen operativo)

Entidades principales

- **users**: perfil, plan, onboarding, permisos, gamificación
- **audiobooks**: mentorías y audiolibros (`contentType`), URLs, métricas
- **books**: metadatos + `etapas[]` con URLs de audio

- **speakers:** Startalent, foto, rol, correo vinculado
- **events:** calendario de eventos/zoom
- **editorial_calendar:** programación semanal/mensual por contenido
- **config/global:** parámetros globales
- **payments:** liquidaciones
- **usage_events:** telemetría
- **naya_memory:** memoria de chat IA
- **userPlaylists:** playlists por usuario
- **success_paths:** rutas y niveles
- **talent_notifications:** reconocimientos/notificaciones a talento

8) Conclusiones de análisis (sin aplicar fixes)

1. El proyecto está correctamente montado sobre **React + Vite + Firebase**.
2. La carga de contenido depende de Firestore/Storage y del modelado correcto (`contentType` , URLs, plan).
3. Hay componentes críticos que aún consumen **mocks** en lugar de datos reales (caso claro: **Calendar**; también coexistencia mock/dinámico en Startalent).
4. La lógica de plan Gratis/Premium está distribuida en varios puntos; existe riesgo de comportamientos no uniformes en onboarding/alta.
5. El botón de instalación PWA sí está implementado, pero su visibilidad depende de condiciones del navegador y del estado persistido en localStorage.
6. No existe módulo de limpieza masiva de datos TEST, solo borrado manual por entidad.

9) Índice rápido de archivos críticos por problema

- **P1 Libros no cargan:**
 - `src/components/Books.tsx`
 - `src/components/BookDetail.tsx`
 - `src/services/dbService.ts` (bookService)
 - `src/components/AdminPanel.tsx` (handleSaveBook)
- **P2 Mentorías no cargan:**
 - `src/services/dbService.ts` (audioService)
 - `src/components/AdminPanel.tsx` (handleSaveMentoring)
 - `src/components/Home.tsx`
 - `src/App.tsx`
- **P3 Fotos Startalent no cargan:**
 - `src/components/AdminPanel.tsx` (handleSaveTalent)
 - `src/services/storageService.ts`
 - `src/components/HallOfFame.tsx`
 - `src/components/StarTalent.tsx`

- `src/components/StarTalentWall.tsx`

- **P4 Eventos calendario no cargan:**

- `src/components/Calendar.tsx`
- `src/services/dbService.ts` (eventService)
- `src/App.tsx`

- **P5 Gratis vs Premium en nuevos usuarios:**

- `src/components/Login.tsx`
- `src/App.tsx`
- `src/services/userService.ts`
- `src/components/AdminPanel.tsx`

- **P6 Botón “descargar app” no aparece:**

- `src/components/InstallPWA.tsx`
- `src/components/Sidebar.tsx`
- `src/App.tsx`
- `src/registerSW.ts`
- `public/manifest.json`
- `public/sw.js`
- `vite.config.ts`

- **P7 Limpieza de datos de prueba TEST*:**

- `src/components/AdminPanel.tsx`
- `src/services/dbService.ts`
- (sin script masivo dedicado en repo)